Exact Neural Network Inference Inside the Weights of Qwen Concept 27B

An RV32IM + Zba processor compiled into 11 transformer layers, verified against a reference emulator

Concept Country
New York, NY · Working paper v1, 8 November 2026

Abstract. A language model can contain a working computer: not a learned approximation of one, but an exact instruction-set implementation whose every state transition is verified against a reference emulator. We port Qwenputer, by Divyansh Agrawal, from Qwen3.5-4B to a 27B hybrid-attention reasoning model, compiling an RV32IM + Zba machine with a bignum coprocessor into 11 transformer layers placed in front of Qwen Concept 27B. The machine's memory is the Gated DeltaNet recurrent state: 256 KB of code ROM, 128 KB of rodata and 32 KB of RAM live in a linear-attention cache. In standalone free-running verification, 2.65 M instructions across 72 programs produce exactly the reference token stream, and a single end-to-end sweep on the merged checkpoint runs 61 programs and 5.09 M instructions with 61/61 matching the reference emulator and 0 divergences. We then compile two neural networks to this instruction set. An MNIST classifier's forward pass runs inside the weights and matches the reference emulator exactly over 2.06 M instructions. A character-level GPT with attention, softmax and a residual stream reproduces its integer specification as a compiled program, and on the merged checkpoint its 23.6 M instructions also match the reference emulator exactly. Both are integer end to end because the machine has no FPU, a design the measurements force rather than suggest: an `expf` costs 517.8 int8 multiply-accumulates, so a float softmax in the inner model would cost 1.12× every matrix multiplication it contains. The host model also calls the machine through its own native tool-call syntax: asked for a product, it emits a `calc` tool call, the in-weights machine computes 121,932,631,112,635,269 and writes the tool response, and the model reads its own answer back. No tool runs outside the weights. The host model's parameters survive the merge intact: 722 of 722 tensors byte-identical by SHA-256 after unpadding, with 129 RMSNorm gains matching a documented rescale exactly and 0 unexpected nonzero padding entries. We report prominently that 6 of 7 pre-specified acceptance criteria are met and one is not: strict greedy text identity against the unmodified host holds on only 16 of 20 prompts at 48 tokens. The cause is identified (bf16 re-rounding of the rescaled norm gains) and bounded: teacher-forced top-1 agreement is 955/960 against a harness self-agreement floor of 954/960, so the discrepancy sits at the measurement noise floor rather than above it, but a criterion that says identical and measures 16/20 has not been met, and we do not reinterpret it.

1. Introduction

A transformer layer is a sequence of linear maps with a nonlinearity between them. A digital circuit is also a sequence of linear maps with a nonlinearity between them, if you are willing to call a lookup table a nonlinearity and a one-hot vector a wire. The observation is old and usually left as an observation, because the interesting direction, compiling a real instruction set into real weight matrices and then checking every state transition against a reference, is a great deal of work for a result that looks like a stunt.

Divyansh Agrawal did that work. Qwenputer compiles an RV32IM + Zba machine, with a bignum coprocessor and a text bridge, into the weight matrices of Qwen3.5-4B. The circuit is exact: it is not trained to imitate a CPU, it is constructed so that its forward pass **is** the CPU's state transition, and its outputs are compared token-for-token against a reference emulator. The machine's memory is the host architecture's own linear-attention state, which is the part that makes the construction fit a transformer rather than fight it.

This paper reports a port, and the port is the contribution. We take that machine to a 27B hybrid reasoning model and merge it into an **edited** checkpoint: Qwen Concept 27B, the subject of our companion paper (Gradient-Free Weight Editing Reduces Overthinking in Qwen3.8-27B, Bennington 2026, Concept Research), which is itself Qwen3.8-27B carrying a rank-one weight-space edit. The question the port answers is not whether a computer can live in weights; Divyansh Agrawal established that. It is whether the construction

survives a 5× scale increase, a different layer-type mix, and a host model whose weights have already been modified for unrelated reasons, and whether the host survives having a computer bolted to its front.

Three results follow. The machine is exact at the new geometry, over 2.65 M instructions. The host's weights are preserved bit-for-bit, which we verify per tensor rather than assert. And the host's **behaviour** is preserved only to within bf16 rounding, which is close enough to be invisible in aggregate and not close enough to satisfy a criterion written as "identical". We report that failure as a failure.

2. Related work

Two lines of work meet here and neither quite contains the result.

The first is weight-space programming: obtaining a direction from a model's activations and writing it into the weights so a behaviour is compiled in rather than applied at inference. Our companion paper replicates that method on the host model used here, and the distinction worth carrying forward is that such edits are **statistical**: a rank-one additive term that shifts a distribution, validated by benchmark deltas against a norm-matched random control. The edit changes what the model tends to do.

The second is the literature on transformer expressivity: constructions showing that attention can implement pointer chasing, that chain-of-thought extends what a fixed-depth model can compute, that specific architectures are Turing-complete given unbounded precision and tape. These results are existence proofs. They establish that a weight configuration exists; they do not produce one that runs `gcc -O2` output.

Qwenputer sits in neither place. It is not statistical; there is no distribution to shift, only a state transition that is either exactly right or wrong, checked at every step. And it is not an existence proof: the machine is compiled, instantiated in bf16, and executed. The closest useful analogy is not machine learning at all but a soft-core processor: an FPGA bitstream describes a CPU in reconfigurable logic, and this describes one in reconfigurable linear algebra. The economics are terrible and the fidelity is exact, which is the opposite of the usual trade in this field.

What makes the host architecture suitable is specific and worth stating. Gated DeltaNet is a linear-attention mechanism carrying a matrix-valued recurrent state updated by a delta rule. With the decay set to retain and the keys one-hot, that state stops being a compressed summary of context and becomes addressable memory: head × key selects a slot, the delta rule writes, a matching query reads. The machine does not simulate RAM in a cache; the cache **is** the RAM, with the write semantics a store instruction needs. A pure softmax-attention stack has no comparable primitive, and this is why the construction is tied to hybrid architectures rather than being general.

3. Method

3.1. What is being merged into what

The host is Qwen Concept 27B, a 64-layer hybrid model with hidden width 5,120, 48 Gated DeltaNet layers and 16 full-attention layers. Its provenance matters for reading our results: it is Qwen3.8-27B plus the rank-one edit characterised in the companion paper, and the merge source is that edited checkpoint (`e_Bcc_0.1`), not stock Qwen. Every claim below about preserved weights is a claim about preserving the **edited** model.

The machine occupies 11 additional Gated DeltaNet layers in front of the host's stack, giving 75 layers total. The compiled circuit uses 2,918 intermediate dimensions, 103 DeltaNet heads and 42,114 MLP units, distributed across those 11 layers with at most 8,149 units and 16 heads in any one.

3.2. Widening without disturbing

The circuit needs residual dimensions the host does not have, so the hidden width grows from 5,120 to 6,656. This is the step that could destroy the host, and the construction avoids it as follows. Every original tensor is zero-padded to the new width and never reads or writes a chip dimension. On chip tokens the host's own

layers are muted through a padding column: DeltaNet's decay, write and output-gate projections are driven to −1024, and full attention additionally receives an anti-sink key, refit on this host.

One class of tensor cannot be left alone. RMSNorm averages over the hidden dimension, so widening changes its denominator. The 129 full-residual norm gains are therefore rescaled by

$$g' = (1 + g) \cdot \sqrt{5{,}120 \;/\; 6{,}656} - 1$$

which leaves the host's normalized activations unchanged up to one bf16 rounding. That rounding is the entire source of the behavioural discrepancy in Section 4.4. It is not an implementation defect; it is the arithmetic consequence of expressing a wider residual stream in a checkpoint format whose norms must span the full width.

3.3. Verification strategy

Three independent checks, because the ways this can go wrong are independent.

Weight preservation is checked per tensor: un-pad each original tensor from the merged checkpoint, hash it, compare to the host's bytes. Pass means byte identity, not approximate agreement. The rescaled norms are checked against the formula rather than against the original bytes, and the padding regions are checked for unexpected nonzeros.

Machine exactness is checked against a reference RV32 emulator at two levels. Teacher forced, the circuit is fed the reference trace and every predicted token compared. Free running, the circuit generates its own trace for thousands of instructions with no correction, and the whole stream is compared. The second is the real test: teacher forcing cannot detect an error that would compound.

Host behaviour is checked by greedy decoding 20 prompts that do not invoke the machine, 48 tokens each, on both the merged checkpoint and the unmodified host, and comparing token sequences. Because that test is a near-tie detector rather than a behaviour test, we additionally measure teacher-forced top-1 agreement and KL on the host's own continuations, and measure the **harness's own floor** by running the host against itself through two different code paths. Without that floor the agreement number is uninterpretable.

3.4. Reproducibility of the numbers in this paper

Every statistic printed here is generated. `emit_results.py` reads the 17 measurement artifacts and writes `results/results.json`; `emit_numbers.py` turns that into the definitions this document interpolates. No figure in the prose is transcribed. The artifacts are vendored with their SHA-256 digests, so a re-run against changed inputs is detectable.

4. Results

4.1. The machine is exact

Check	Programs	Instructions	Result
Teacher-forced IR suite	—	—	237 passed / 0 failed
Text bridge, host tokenizer	—	—	9 passed
Free-running fuzz	64	152,848	exact
Free-running C (`gcc -O2`)	8	2,495,296	exact
Total free-running	72	2,648,144	all exact

Table 1: Circuit exactness at the 27B geometry. The teacher-forced suite covers riscv-tests rv32ui / rv32um / rv32uzba, compiled C, faults, CSRs, and ALU / MUL / DIV / branch fuzzing. Free-running runs generate their own traces with no correction and are compared token-for-token against the reference emulator.

2,648,144 instructions across 72 programs produce exactly the reference token stream. The teacher-forced suite passes 237 of 237 checks. The upstream reproduction at Qwen3.5-4B, run first as a control on our hardware,

gave 239 passed / 1 failed / 14 skipped; the single failure was an upstream bug in the free-running executor (`KeyError: 'Q'` in `test_fuzz_free_running_cpu`), fixed by a two-line patch, after which the suite is green. We report it because it is the one place our numbers required changing someone else's code.

4.1.1. A single end-to-end sweep

The figures above come from the build's phase-2 log. To put the exactness claim on evidence the reader can recompute, we then ran one corpus end to end on the merged checkpoint and retained a JSON record per program. Every number in this subsection is derived from those 61 records by `emit_results.py`, not transcribed.

Check	Programs	Instructions	Result
riscv-tests ISA	51	—	51/51 exact
C programs (`gcc -O2`)	10	—	10/10 exact
Sweep total	61	5,094,103	61/61 exact

Table 2: The RV32IM sweep on the merged checkpoint. 0 programs diverged, 0 errored and 0 failed to halt; the longest single program, `c-sudoku`, executes 2,580,085 instructions.

The sweep is graded on the circuit's own token stream. A merged bridge checkpoint echoes every printed byte back as a host text token by design, so the stream a reader observes contains tokens the reference trace does not, and raw token equality fails for any program that prints: it holds for 55 of 61 programs, against 61 of 61 on the circuit's stream with echoes removed.

Three properties of the records establish that the excess tokens are echoes rather than drift. The partition is exact: of 61 programs, the 6 that print are exactly the ones failing raw equality (0/6), and all 55 non-printing programs pass it (55/55). Token excess equals echo count program by program, with no exceptions across the corpus. And every echo is the host text token of the byte the chip had just printed, giving 0 integrity failures across 189 echoes.

The distinction between the circuit's stream and the stream a reader observes is a property of in-weights machines with a text bridge generally, not of this build.

4.2. The host model calls the machine and reads its own answer

Asked *Use calc: what is 123456789 * 987654321?*, the merged checkpoint emits its native tool-call syntax for `calc`, the in-weights machine executes the multiplication and writes `<tool_response>121932631112635269</tool_response>`, and the host then answers 121,932,631,112,635,269 in prose. The exchange costs 119 text tokens and 2,146 machine tokens. Replaying the generation and checking every circuit-owned position gives 0 wrong predictions and 0 ownership mismatches across 2,176 owned positions out of 2,671 total.

Nothing in this runs outside the weights. There is no interpreter, no sandbox, no function dispatch to a host process: the tool call and the tool response are both produced by matrix multiplications in the same forward pass.

Program	Instrs	Tokens	Ref tokens	Machine stream	tok/s
`crc32`	2,665	4,165	4,165	exact	262
`strops`	707	1,073	1,073	exact	261
`mini-gmp` multiply	14,767	22,715	22,670	exact[†]	268

Table 3: C programs run free-running through the merged checkpoint's own stock HuggingFace chip modules in bf16. The machine instruction stream is exact for all three. [†] `mini-gmp`'s machine stream is exact and its printed product correct, but the run emits +45 tokens against the reference total, so its end-to-end exactness flag is false. 2 of 3 programs are token-exact end to end; 3 of 3 have exact machine streams.

`mini-gmp` computes a 20-digit by 20-digit bignum multiplication in 14,767 instructions and prints 121932631137021795224980948001249809479O, with each printed byte echoed back as a host text token, 45 of 45 correct. The run emits 22,715 tokens against 22,670 reference tokens, first differing at position 21,725, where an extra text echo appears in the tail. The machine stream is exact and the printed product is correct, so the computation is right and the difference is in how the tail of the text bridge is emitted.

4.3. Neural network inference inside the machine

The sweep establishes that the machine runs the corpus it was built against. A stronger question is whether it runs programs its construction did not anticipate, with access patterns unlike anything in that corpus. We compiled two neural networks to RV32IM + Zba and ran their forward passes on the merged checkpoint: a neural network computed by a CPU that is itself a neural network's forward pass.

Both inner models are integer end to end, and that is a measurement rather than a preference. The ISA has no FPU, so every float operation becomes a soft-float libgcc call. We measured the cost on the machine itself with a micro-benchmark whose baseline loop is subtracted, counting instructions on the reference emulator over 10,000 iterations:

Operation	Instructions	Relative
int8 multiply-accumulate	4.22	1.0×
float32 divide	183.59	43.5×
float32 multiply-accumulate	189.38	44.9×
`sqrtf`	337.68	80.0×
`expf`	2184.94	517.8×

Table 4: Instruction cost per operation on the in-weights machine, measured rather than assumed. An `expf` costs 517.8 int8 multiply-accumulates.

These numbers decide the design. A faithful float softmax in the inner transformer below would issue 256 `expf` calls per token at full context, costing 559,345 instructions against 501,539 for every matrix multiplication in the model combined: a ratio of 1.12. The nonlinearity would cost more than the model. So the inner models use int8 weights with int32 and int64 accumulators, requantization by a calibrated fixed-point multiplier (`((acc * M0) >> shift`, two instructions given RV32M's `MULH`), RMSNorm with an exact integer floor-sqrt, and softmax by a 256-entry exponential lookup table. Weights are `const` and live in rodata, so the scarce 32 KB of RAM holds only activations.

4.3.1. A classifier

The first inner model is an MNIST classifier, 784-32-10 MLP, ReLU, with 25,408 int8 parameters occupying 33 KB of rodata. Quantization costs 0.03 percentage points of test accuracy: 95.29% in float32, 95.26% in int8 over the full test set. The program classifies 10 images and prints the winning logit alongside each prediction, which makes the output sensitive to every accumulator bit rather than letting a wrong multiply hide behind a still-correct argmax.

On the merged checkpoint the machine executes 2.06 M instructions and the machine's token stream matches the reference emulator exactly (yes), with 322 echoes all matching their printed bytes. All 10 of 10 predictions are correct and every printed logit is identical to the PyTorch reference.

4.3.2. A transformer inside a transformer

The second inner model is a character-level GPT: char GPT V=65 d=64 L=2 H=2 ff=256 ctx=64, trained on tiny Shakespeare, with 110,976 int8 parameters in 110 KB of rodata. It has the same architectural ingredients as its host, attention, softmax, a residual stream and autoregressive decode, and it is compiled for the machine the host's weights implement.

Quantization is calibrated, not guessed. 18 activation tensors are profiled at their 99.9th percentile over real batches and 20 fixed-point multipliers derived from the observed ranges. Guessing shift constants instead gave an integer validation loss of 27.6 against the float model's 1.877. With calibration the integer model reaches 1.663, slightly better than float; we did not investigate why. Greedy continuation of `ROMEO:` agrees between the float and integer models for 26 characters:

```
float32   ROMEO: ⏎ What the see the see th
int8      ROMEO: ⏎ What the see the seasur
```

after which one tie-break differs and greedy decode amplifies it.

The expected token stream is derived from the emitted weight header rather than from the training process, and the distinction is not pedantic. The trainer holds float scales and calibration tables in memory; only the int8 arrays and the multiplier pairs ship. While the specification came from the trainer's in-process path, the compiled program disagreed with it at the second generated token even though a stage-by-stage trace showed the two agreeing bit for bit on the first decode step. Re-deriving the specification from the header, using exactly the arithmetic the C performs, is what made them agree.

Each generation step replays the whole prefix, so the instruction count grows quadratically with the number of generated tokens. Compiled for RV32IM + Zba, the program reproduces its specification token for token under QEMU. On the merged checkpoint a 4-token generation executes 23.6 M instructions through the Triton executor, and the machine's token stream against the reference emulator is graded exact: yes. The executor is a different path from the stock HuggingFace chip modules the sweep used, so we ran the MNIST program through both: they agree on instruction count, output and verdict (yes), with the executor 6× faster.

Inner model	Params	Instructions	Machine stream	tok/s
MNIST MLP	25,408	2,059,619	yes	259
MNIST MLP (executor)	25,408	2,059,619	yes	1624
Character GPT, 4 tokens	110,976	23,600,604	yes	5190

Table 5: Neural network inference on the merged checkpoint. The grading metric is the machine's own token stream against the reference emulator, with printed-byte echoes checked and excluded as in the sweep.

4.3.3. Quantizing the chip layers breaks the machine

A text model degrades gracefully under quantization. The chip layers are not a text model, so we tested them separately. We rounded the 50 chip-layer weight tensors that a standard recipe would quantize onto the weight grids of 3 lossy formats (FP8 block, and llama.cpp's 8-bit and 4-bit block formats), leaving the 104 tensors such recipes keep in bf16 untouched, and ran 4 programs after each. The bf16 round-trip control stays exact on all of them (yes). Of the 3 lossy grids, 0 keep the machine exact, and 0 program runs in total survive. The gentlest grid that breaks it perturbs a weight by at most 0.39% of its tensor's largest magnitude. This simulates weight grids only, not a quantized inference kernel, so it shows that these formats cannot be applied to the chip layers, and says nothing yet about a checkpoint that quantizes only the host's layers and keeps the chip layers in bf16.

What this shows is that the processor in the weights is a general computer: it runs a kind of program its test corpus never exercised, and the classifier's result is bit-exact against an independent emulator. What it does not show is any benefit to the host. A conventional CPU performs this inference in microseconds. The result is computational universality, not performance.

4.4. One criterion is not met

The build was specified against seven acceptance criteria. 6 are met. The one that is not is text identity against the unmodified host.

Criterion	Result
Upstream 4B reproduced (riscv-tests green, calc exact)	met — 239 passed / 1 failed / 14 skipped; the single failure was an upstream executor bug, patched
27B loads as stock HF; Qwen tensors byte-identical	met — 722/722 identical, 129 norm gains match the documented rescale exactly
Circuit exact at 27B, teacher-forced riscv-tests + fuzz	met — 237 passed / 0 failed; 2.65M instructions free-running exact
End to end: exact product via in-weights tool call	met — 121932631112635269; 2,176 owned positions replayed, 0 wrong
End to end: one C program	met — crc32 and strops token-exact; mini-gmp chip stream exact with a trailing text-echo discrepancy
20 prompts identical greedy vs Concept	not met — 16/20 identical at 48 tokens; teacher-forced top-1 955/960 against a harness floor of 954/960
Throughput measured on GB10	met — 5,891 trace tok/s batch 1, 23,467 batch 32

Table 6: All seven pre-specified acceptance criteria, generated from the results file so the verdicts cannot drift from the measurements.

Greedy decoding 20 prompts for 48 tokens gives 16 identical continuations and 4 divergent ones (prompts 1, 16, 17, 18), first diverging at token positions 0, 13, 31, 41. Both versions stay fluent and on topic where they differ. The criterion said identical on 20 prompts. 16 is not 20.

What we can establish is the size of the effect. Teacher-forced on the host's own continuations across 960 positions, top-1 agreement is 955/960 with mean KL 5.5e-4 and max 3.1e-2. Running the **host against itself** through the two code paths the harness uses, prefill versus incremental decode, gives 954/960. The merged checkpoint therefore agrees with the host +1 positions better than the host agrees with itself under a change of kernel path. The worst single prompt (16) scores 46/48, and it scores 46/48 on the self-comparison too.

The honest reading is that the discrepancy is at or below the measurement floor, and that the floor is not zero. Greedy decoding is a near-tie amplifier: a logit shift of about one ulp flips an argmax wherever two tokens are close, and the flip then propagates. The upstream project reports the same picture at Qwen3.5-4B, KL 6e-4 against a 5e-4 floor.

This does not rescue the criterion. Exact identity would require RMSNorm that ignores the padding dimensions, which the stock checkpoint format cannot express; the alternative is to restate the criterion as a KL bound, which is what upstream does and which we decline to do retroactively. The criterion as written is not met, and the correct summary of this build is 6 of 7.

4.5. Throughput

Path	Trace tok/s	Note
Triton executor, batch 1	5,891	3,769 instr/s
Triton executor, batch 8	19,866	aggregate
Triton executor, batch 32	23,467	15,015 instr/s aggregate
`torch.compile` executor, batch 1	861	same circuit
Stock HF chip modules, batch 1	262	no custom runtime
Qwen3.5-4B in chat, executor	237	end to end, both models resident
27B in chat, host streamed from disk	0.7	I/O bound — not representative

Table 7: Measured on one DGX Spark (GB10, aarch64). All executor rows verified exact. The batch-1 executor is 22× the stock-module path; batching to 32 gives 4.0× aggregate throughput over batch 1.

The last row needs disclaiming rather than quoting. A full-speed 27B chat run needs about 90 GB of accelerator memory, which was not available while production work occupied the host; the model's own layers were streamed from disk, and 0.7 tok/s measures that I/O path, not the model. The single `calc` exchange took 50 minutes for this reason. We report it for completeness and attach no meaning to it. The honest statement about 27B chat throughput is that it is unmeasured.

4.6. Where the parameters go

Component	Params
Qwen Concept 27B at native width 5,120	26.90 B
+ zero-padding those 64 layers to 6,656	8.07 B
+ 11 chip layers (the machine)	5.48 B
+ machine vocabulary rows (embedding and head)	3.07 B
merged checkpoint	43.51 B

Table 8: Parameter decomposition, computed from the actual layer-type sequence with Gated DeltaNet and gated-attention shapes. The components sum to the total (closure: yes). Model validated against the checkpoint's known tensor count (1,005 predicted vs 1,005 reported: yes).

The headline growth from 27B to 43.51 B is mostly not information. 8.07 B, 18.5% of the checkpoint, is zeros, present only because a wider residual stream must be expressed in a format that stores every tensor at full width. The machine itself is 5.48 B of live parameters plus 3.07 B of vocabulary rows for its trace tokens.

59 of the 75 layers are Gated DeltaNet, whose projections are shaped differently from standard multi-head attention, and `attn_output_gate` is true, which fuses a sigmoid gate into `q_proj` and doubles its output width. The gate shares the `q_proj` tensor, so it contributes 0.65 B parameters without changing the tensor count: a decomposition without it gives 42.86 B against the measured 43.51 B.

The hidden size grows from 5,120 to 6,656. Of the added dimensions, 26 are ballast, 1 is tau and 1,265 are chip slots, 1,292 in all; the remaining 244 bring 6,656 to 26× the head dimension. The vocabulary is 248,320 host rows plus 230,324 machine rows, 478,644 in all, stored as 478,656, a multiple of 64.

The checkpoint is 87.0 GB (81.0 GiB) in bf16.

It loads as a stock `Qwen3_5ForCausalLM` with 1,005 tensors, 0 missing, 0 shape mismatches and 0 unexpected keys. No custom runtime is required for correctness. The custom decoder exists only for speed: it lets machine tokens skip the host's 64 layers, which plain `generate()` would otherwise run for every one of the 2,906,158 trace tokens.

4.7. The host's weights survive

Check	Count
Tensors byte-identical after un-padding (SHA-256)	722 / 722
RMSNorm gains matching the documented rescale exactly	129 / 129
Mismatches	0
Missing tensors	0
Padding entries unexpectedly nonzero	0
Vision / MTP tensors dropped (text-only build)	348

Table 9: Per-tensor preservation of the host model. 851 tensors checked. Byte identity, not approximate agreement.

Every one of the host's 722 carried tensors is byte-identical to the source checkpoint, and every one of the 129 tensors that had to change equals the documented transform of the source bytes exactly. The rank-one edit from the companion paper is carried through untouched, because it lives in those bytes.

4.8. The released formats

The released checkpoint is not the one the results above were measured on. The measured build is text-only, 1,005 tensors; the release keeps the vision tower (333 tensors) and the MTP head (15), 1,353 in all, and its 154 of 154 machine-layer tensors are byte-identical to the measured build. The two differ only in that 129 RMSNorm gains are stored float32 in the release rather than bf16. The release layout loads as the stock conditional-generation class with no missing tensors and no shape mismatches.

5 formats are published. In each of the quantized ones the machine's 11 layers, the token embedding and the output head stay bf16, and only the host's layers are quantized. The reason is in Section 4.3: on the weight grids of these formats the machine goes from 4 of 4 programs exact to 0, while the bf16 control stays exact. A format that quantized the machine would be a different artifact, not a smaller one.

Format	Scheme	Machine check	Held	Loads	Text vs base
`bf16`	bfloat16	sha256	yes	yes	18 of 20
`fp8`	FP8_BLOCK, 128x128 blocks	sha256	yes	yes	14 of 20
`nvfp4a16`	NVFP4 weights, group 16, bf16 activations	sha256	yes	yes	10 of 20
`Q8_0`	GGUF Q8_0	dtype	yes	yes	not measured
`Q4_K_M`	GGUF Q4_K_M, from the bf16 GGUF	dtype	yes	yes	not measured

Table 10: The released formats. "Machine check" is how the machine's tensors were compared: sha256 of every tensor in the machine's layers, the embedding and the head against the bf16 release, or, for the GGUF builds, the dtype of every tensor in that same region. "Text vs base" is greedy continuation of 20 prompts for 48 new tokens against the same format of the base model, counting identical continuations.

The machine survived in 5 of 5 formats and every format loads. The GGUF conversion renames, fuses and transposes relative to the HuggingFace layout, so a tensor-by-tensor value comparison across the two is not well defined; what is checked there is that no tensor in the machine's region carries a quantized type, which is the claim being made. The GGUF builds exclude the MTP head, as the base model's own GGUF does.

Text accuracy is measured against the same format of the base model rather than against bf16, because the question is whether the merge costs accuracy, not whether quantization does. At bf16 the release reproduces the base model's greedy continuation on 18 of 20 prompts, with the differences beginning a median of 121 characters into the continuation and reading as paraphrase rather than error. Quantization widens that gap in the ordinary way. Text identity for the two GGUF formats was not measured: the runtime available here is a CPU build and the comparison does not fit the machine's time budget. That cell says so rather than carrying an inferred number.

5. Discussion

The narrow claim is secure and worth stating precisely. An exact RV32IM + Zba machine, with 256 KB of code ROM and 32 KB of RAM held in linear-attention state, runs inside the weight matrices of a 27B reasoning model, executes 2.65 M instructions identically to a reference emulator across the phase-2 suite and a further 5.09 M across the 61-program end-to-end sweep, and is invoked by the host model through the host's own tool-call syntax with no external interpreter. The host's parameters are preserved byte-for-byte. This is a port of Divyansh Agrawal's construction and the construction is his; what we establish is that it survives the scale change, the different layer mix, and an already-edited host.

The interesting consequence is about what a tool call can be. A model that calls an external calculator depends on a runtime contract: the tool exists, is reachable, returns what it claims. Here the contract is the checkpoint. The arithmetic is as portable as the weights, and it cannot be removed without removing parameters. Whether that is useful is a separate question: 262 tok/s through stock modules for a machine that a CPU would run billions of times faster is not a performance argument. It is an argument about where a capability **lives**.

The contrast with the companion paper is the cleanest way to see what is unusual. That work writes a direction into nine matrices and shifts a distribution; it is validated statistically, against a norm-matched control, because the effect is statistical. This work writes a state machine into eleven layers; it is validated by exact comparison, because there is nothing to average. Both are weight-space programming. One compiles a tendency and the other compiles a program, and the second needs no control arm because an emulator disagreement is not a matter of degree.

What this does not establish is that the approach generalises beyond hybrid linear-attention architectures. The construction depends on Gated DeltaNet state being addressable memory. It does not establish any benefit to the host model: we measured that the host is unharmed, not that it is improved, and the only tool wired to the machine is `calc`. And it does not establish throughput at 27B in a realistic serving configuration, because we could not fit the model.

6. Limitations

The failed criterion is the first limitation and we have stated it plainly: greedy text is not bit-identical to the host, 16 of 20 prompts at 48 tokens, caused by bf16 re-rounding of the rescaled norm gains. The KL evidence bounds the effect at the measurement floor; it does not satisfy the criterion. Exact identity needs norms that ignore padding dimensions, which the stock format cannot express.

Full-GPU throughput for the 27B merge is unmeasured. The figure in our speed table for 27B chat (0.7 tok/s) measures disk streaming on a host with insufficient free memory and should not be cited as a throughput result. The machine needs about 90 GB to run resident, and the host that would have provided it was down at the time of writing and remains so, which is also why no further measurement was attempted for this draft.

The text-identity evaluation is 20 prompts at 48 tokens, which is a small sample for a near-tie detector; the teacher-forced comparison is 960 positions on the host's own continuations and therefore measures agreement on a distribution the host generated, not on held-out text. We did not evaluate the merged checkpoint on any capability benchmark, so the claim "the host is unharmed" rests on token agreement and weight identity, not on task performance. A reader who wants to know whether Qwen Concept 27B's benchmark results from the companion paper survive the merge should treat that as untested.

The machine's correctness is established against one reference emulator. A bug shared between the compiler and the reference would be invisible to every check here. The riscv-tests suites constrain this but do not eliminate it.

Numerics are a standing hazard: the chip layers require exact bf16 matmul accumulation (`allow_bf16_reduced_precision_reduction = False`), and a default-configured runtime will produce wrong results silently. The bundled scripts set it; a third party reproducing this must.

The build is text-only (the host's vision tower is not carried over) and the host checkpoint itself is not final pending its own evaluation verdict, so the merge may be rebuilt. Qwen Concept 27B RV32IM 27B is experimental and under evaluation. Nothing here is a release.

Finally, the one-paragraph caveat on the whole exercise: we are reporting a port of someone else's construction, measured on our own hardware with our own host model. The parts that would be most valuable to a reader, why the circuit is correct and how the compiler lowers RV32 semantics into attention, are documented in Qwenputer, not here.

Status. Working paper v1. 6 of 7 pre-specified acceptance criteria met; 1 not met (strict greedy text identity, 16/20). The model, Qwen Concept 27B RV32IM 27B, is experimental and under evaluation.

Model availability. The merged model is published as `conceptcountry/concept-rv32im-27b` on models.concept.country and on HuggingFace, in bf16 and FP8. FP8 is what brings it within a single 128 GB

accelerator; bf16 is what every measurement here used. Both carry the vision tower and the multi-token-prediction head, which an earlier merge dropped.

Data and code. The 17 measurement artifacts are vendored with SHA-256 digests alongside `emit_results.py`, which recomputes every statistic, and `emit_numbers.py`, which generates the definitions this document interpolates, at github.com/conceptcountry/concept-rv32im, Apache-2.0 as the upstream work requires. No number in this text is transcribed by hand.

Acknowledgements. Qwenputer is the work of **Divyansh Agrawal** (https://github.com/divagr18/qwenputer): code MIT, weights Apache-2.0. The circuit, the compiler, the RV32 firmware and the text bridge are his; its internal architecture is called chipformer. Our contribution is fitting the compiler to the 27B geometry, merging into Qwen Concept 27B, and the verification reported here. Our model name reuses neither Qwenputer nor chipformer, so the credit is stated in prose instead: this is a port, and the original is his. The base model is Qwen3.8-27B (Qwen team, Apache-2.0). The host edit and its evaluation are described in the companion paper (Gradient-Free Weight Editing Reduces Overthinking in Qwen3.8-27B). The analysis, figures and text are the authors'.